DBMS LAB · ASSIGNMENT 3

Database Design Analysis and Normalization

Redundancy, anomalies and functional dependencies in the Airline Reservation & Flight Booking design

STUDENT
Reddy Sai Abhiram

GITHUB
github.com/reddysaiabhiram2468

PROJECT
Airline Reservation & Flight Booking System

DATABASE
MySQL 8.0 · `airline_reservation`

SUBMITTED
25 October 2026

Introduction

The Airline Reservation & Flight Booking System manages flight schedules, aircraft inventory, passenger profiles, ticket bookings, seat assignments and payment records for an airline. The design proposed in the previous assignment uses six tables:

Table	Proposed columns
Passengers	passenger_id (PK), first_name, last_name, email (UNIQUE), phone_number, passport_number (UNIQUE), nationality
Airports	airport_code (PK, CHAR(3)), airport_name, city, country
Flights	flight_id (PK), flight_number, origin_airport_code (FK), destination_airport_code (FK), departure_time, arrival_time, total_seats, base_price
Bookings	booking_id (PK), passenger_id (FK), booking_reference (UNIQUE), booking_date, total_fare, status
Ticket_Details	ticket_id (PK), booking_id (FK), flight_id (FK), seat_number, travel_class, fare
Payments	payment_id (PK), booking_id (FK, 1 : 1), payment_method, payment_status, transaction_date

This report checks the design for redundancy and for insertion, deletion and update anomalies. It derives the functional dependencies of the airline's data, normalizes the data step by step, and proposes the improvements that make up the **finalized design** implemented in Assignments 5 to 9.

1. What Goes Wrong in a Single Booking Sheet

Many small travel agencies keep a single booking sheet with one row per ticket and every detail written out. As a table, `BookingSheet` looks like this (one row per passenger per flight):

pnr	passenger	passport	flight	date	from	to	dep	aircraft	seats	class	seat	fare	paid
K7QF2M	Srinivas Reddy	M4821937	SW101	02-Oct	HYD	DEL	06:10	A320neo	180	Economy	14C	6,210	UPI
K7QF2M	Lakshmi Reddy	N1937285	SW101	02-Oct	HYD	DEL	06:10	A320neo	180	Economy	14D	6,210	UPI
K7QF2M	Srinivas Reddy	M4821937	SW102	09-Oct	DEL	HYD	09:30	A320neo	180	Economy	21A	5,880	UPI
P3XW8A	Ananya Mehta	R2208471	SW101	02-Oct	HYD	DEL	06:15	A320neo	186	Business	3A	19,870	Card
T9HN4C	Kiran Kumar	M4821937	SW201	05-Oct	HYD	BOM	07:40	A320neo	180	Economy	14C	4,410	–

Shaded cells conflict with other rows: the same flight with two departure times and two seat counts, and a passport number shared by two different people.

1.1 Redundancy

- The origin, destination and departure time of flight SW101 are written on every ticket sold for it, and again for every date it operates.

- The number of seats depends on the aircraft type, yet it is copied onto every row of every flight.
- A passenger's name and passport are repeated on every ticket they ever buy.
- The PNR's payment method is repeated on each ticket of the booking.

1.2 Anomalies

Anomaly	Example	Why
Insertion	A new route (Hyderabad–Kochi) or a newly delivered aircraft cannot be recorded until a ticket is sold on it. A registered customer with no booking cannot be stored at all.	Routes, aircraft and passengers exist only as parts of ticket rows.
Deletion	Removing the only ticket on a flight also removes the fact that the flight was scheduled. Deleting a cancelled booking can erase the only record of a passenger.	Unrelated facts are deleted together with the ticket.
Update	When SW101's departure moves from 06:10 to 06:15, or the aircraft is changed to one with 186 seats, every row for that flight must change. Row 4 shows a partial update: two times and two capacities for one flight.	One fact is stored in many rows.

How poor design causes these anomalies: the sheet describes passengers, airports, routes, aircraft, dated flights, bookings, seats and payments in a single relation. Its key identifies one ticket, but most columns depend on something else (the flight, the aircraft, the passenger or the booking). Columns that do not depend on the key repeat and drift apart. Normalization gives each of those things its own table.

2. Functional Dependencies

	Dependency	Reason
F1	`passenger_id` ⟶ `first_name, last_name, email, phone, passport_number, nationality, date_of_birth`	Passenger profile
F2	`email` ⟶ `passenger_id`; `nationality, passport_number` ⟶ `passenger_id`	Passport numbers are unique only within the issuing country
F3	`airport_code` ⟶ `airport_name, city, country`	IATA code identifies an airport
F4	`city` ⟶ `country`	Transitive inside Airports (discussed in 4.1)
F5	`flight_number` ⟶ `origin, destination, scheduled_departure, duration, distance`	A flight number is a fixed route flown repeatedly
F6	`flight_number, departure_date` ⟶ `flight_id`; `flight_id` ⟶ `flight_number, aircraft, departure_time, arrival_time, base_price, status`	One operation of a route on one day
F7	`aircraft` ⟶ `seats per class`	Capacity depends on the aircraft, not on the flight
F8	`travel_class` ⟶ `fare_multiplier, checked_baggage`	Rules of a cabin class
F9	`booking_reference` ⟶ `booking_id`; `booking_id` ⟶ `passenger_id, booking_date, status`	One PNR
F10	`ticket_id` ⟶ `booking_id, flight_id, passenger_id, travel_class, seat_number, fare, status`	One seat for one traveller
F11	`flight_id, seat_number` ⟶ `ticket_id`; `flight_id, passenger_id` ⟶ `ticket_id`	For live (not cancelled) tickets only
F12	`booking_id` ⟶ `total_fare`	Derived: sum of the fares of its live tickets

	Dependency	Reason
F13	`payment_id ⟶ booking_id, amount, method, status, transaction_ref, date`	One payment attempt

In the proposed `Flights` table the key is flight_id, but F5 means that `flight_id ⟶ flight_number ⟶ origin, destination` is a transitive dependency, and F7 means that total_seats depends on the aircraft rather than on the flight. Likewise, in `Bookings`, total_fare depends on the tickets (F12), not on the booking key alone.

3. Normalization

Step	Rule	Applied to the booking sheet
1NF	Atomic values, no repeating groups	One row per ticket instead of listing several passengers or legs in one cell; the passenger name is split into first and last name.
2NF	No partial dependency on a composite key	With key {pnr, passenger, flight, date}, booking columns depend on the PNR alone, passenger columns on the passenger alone, and flight columns on {flight, date} alone. These become Bookings, Passengers and Flights, and Ticket_Details keeps only seat, class and fare.
3NF	No transitive dependency	Airport details move to Airports (F3). The recurring route moves from Flights to Routes (F5), the capacity moves to Aircraft (F7), and the class rules move to Travel_Classes (F8).
BCNF	Every determinant is a candidate key	The determinants left are the primary keys plus email, (nationality, passport_number), flight_number, (route, date), registration, class_name, booking_reference and transaction_ref, all declared UNIQUE. City → country is a documented exception.

Each split shares a key with the table it came from, so joining the tables reproduces the original sheet exactly (a lossless decomposition). Every dependency can be checked within one table, except F11, which applies only to live tickets. F11 is enforced with UNIQUE keys on generated columns.

4. Review of the Proposed Six-Table Design

#	Proposed design	Problem	Anomaly	Improvement
1	Flights holds flight_number, origin and destination	SW101 flies HYD–DEL every few days. The route is repeated for every dated flight (F5).	Update: changing a route's airport or scheduled time means editing every future flight. Insert: a route cannot exist before its first flight is scheduled.	New **routes** table (flight_number, airports, scheduled time, duration, distance). **flights** keeps one row per dated departure with a route_id.
2	Flights.total_seats	Capacity belongs to the aircraft (F7). The purpose statement mentions “aircraft inventory”, but no table stores it.	Update: swapping the aircraft for a flight leaves a wrong seat count. Seats per class are not known at all.	New **aircraft** table with seats per class. flights.aircraft_id is a foreign key.
3	Ticket_Details.travel_class ENUM	Fare multiplier and baggage allowance depend on the class (F8) but are stored nowhere.	Pricing rules must be hard-coded in the application.	New **travel_classes** table; tickets reference a class_code.
4	Bookings.total_fare	Derived from ticket fares (F12).	Update: cancelling one passenger in a booking leaves a stale total.	Removed. The view **v_booking_totals** computes it.
5	Ticket_Details has no traveller	A booking for a family has one passenger_id (the booker), so there is no record of who sits in seat 14D.	Insert: a booking for several travellers cannot be stored properly. Boarding cannot match passports to seats.	Add **passenger_id** to tickets for the traveller. bookings.passenger_id becomes the person who booked.

#	Proposed design	Problem	Anomaly	Improvement
6	Payments: 1 : 1 with Bookings, no amount	A failed card attempt followed by a successful UPI payment needs two rows, and the amount paid is not recorded.	Insert: a retry cannot be stored. Reconciling payments against fares is impossible.	Payments becomes **1 : N** with **amount** and a unique **transaction_ref**. At most one *successful* payment per booking is allowed (generated-column UNIQUE).
7	Seat assignment	Nothing stops two live tickets with seat 14C on the same flight, or one person booked twice on a flight.	Double-booked seats.	Add **ticket_status**. UNIQUE (flight, seat) and UNIQUE (flight, traveller) apply to live tickets only.
8	Passengers.passport_number UNIQUE	Passport numbers are unique per issuing country, not worldwide (F2).	A valid foreign passenger could be rejected.	UNIQUE (nationality, passport_number); + date_of_birth (needed for child fares and travel documents).
9	Flights.departure_time / arrival_time	Nothing ensures arrival after departure, or that origin and destination differ.	Invalid schedules can be stored.	CHECK constraints, plus a flight_status column (Scheduled, Delayed, Departed, Arrived, Cancelled).

4.1 Accepted or intentional redundancy

- **Ticket_Details.fare** equals base price × class multiplier at the time of sale. Base prices change daily, so the fare actually charged is a historical fact and is kept. A trigger fills it in when it is not supplied.
- **Flights.base_price** stays on the dated flight rather than the route, because airlines price each departure separately (season, weekday, demand).
- **Airports.city → country** is a transitive dependency. A separate Cities table would remove it, but airport data is fixed reference data that rarely changes, so the small redundancy is accepted.

5. Finalized Design

Table	Status	Change from the proposed design
airports	kept	Unchanged.
aircraft	**new**	Registration, model, seats per class, in-service date, status.
travel_classes	**new**	Class code and name, fare multiplier, baggage allowance.
routes	**new**	Flight number, origin, destination, scheduled time, duration, distance (from Flights).
flights	changed	route_id and aircraft_id replace flight_number, airports and total_seats; + flight_status.
passengers	changed	+ date_of_birth; passport unique per nationality.
bookings	changed	total_fare removed (view); status Pending / Confirmed / Cancelled.
ticket_details	changed	+ passenger_id (traveller), class_code FK, ticket_status; live seat and traveller unique.
payments	changed	1 : N; + amount, transaction_ref; one successful payment per booking.

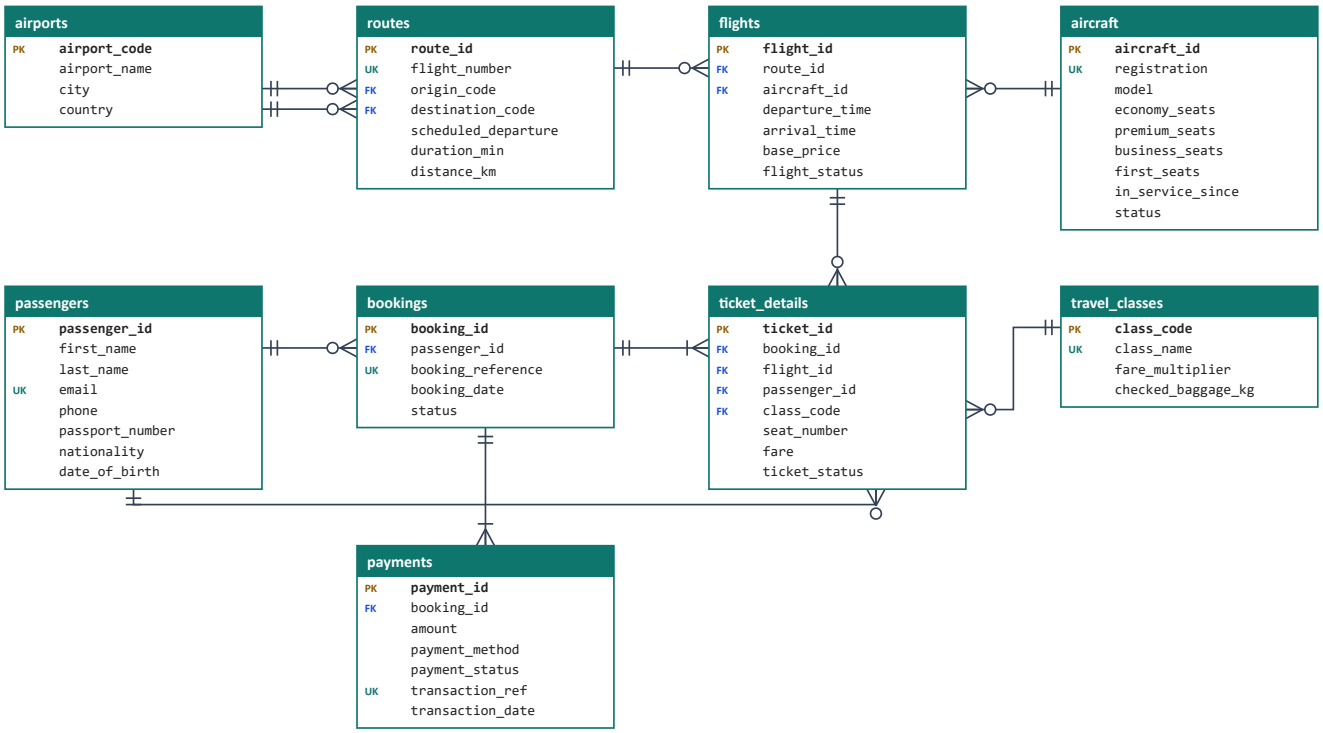


Figure 1. Finalized airline reservation schema (9 tables), implemented in Assignment 5.

6. Conclusion

A single booking sheet repeats flight, aircraft and passenger details on every ticket, which causes insertion, deletion and update anomalies. The functional dependencies of airline data separate eight kinds of facts: passengers, airports, recurring routes, dated flights, aircraft, cabin classes, bookings with their tickets, and payments. The proposed six-table design captured most of them. It still mixed routes with dated flights, stored capacity on the flight, kept a derived booking total, had no record of who occupies each seat, and allowed only one payment per booking with no amount. The finalized nine-table design fixes these points and adds constraints that prevent double-booked seats.